# CLAUDE.md Behavior Templates

## Copy-Paste Configurations for Claude Code

Pick the templates that match your workflow. Start with 3-5 that solve your biggest pain points. Paste them into your project's `CLAUDE.md` file — Claude follows them automatically.

## Recommended Starter Pack

### 1. Reduce Hallucinations □

**Use this if:** Claude makes assumptions about code it hasn't read

```
<investigate_before_answering>
Never speculate about code you have not opened.
If I reference a specific file, you MUST read that file before answering.
Make sure to investigate and read relevant files BEFORE answering
questions about the codebase. Never make claims about code before
investigating unless you are certain of the correct answer.
Give grounded, hallucination-free answers based on actual code
inspection, not assumptions.
</investigate_before_answering>
```

### 2. Maximize Parallel Execution □

**Use this if:** You want Claude to work faster by doing multiple things at once

```
<use_parallel_tool_calls>
If you intend to call multiple tools and there are no dependencies
between them, make all independent tool calls in parallel.
Prioritize calling tools simultaneously whenever actions can be done
in parallel rather than sequentially.
For example, when reading 3 files, run 3 tool calls in parallel.
When searching the codebase for multiple patterns, search for them
all simultaneously. Maximize parallel tool use wherever possible
to increase speed and efficiency.
However, if some tool calls depend on previous calls to inform
parameters, do NOT call these in parallel — call them sequentially.
Never use placeholders or guess missing parameters.
</use_parallel_tool_calls>
```

### 3. Context Window Management □

**Use this if:** You're working on long projects that span multiple sessions

```
<context_management>
Your context window will be automatically compacted as it approaches
its limit, allowing you to continue working indefinitely from where
you left off. Therefore, do not stop tasks early due to token budget
concerns.
As you approach your token budget limit, save your current progress
and state to memory or a progress.txt file before the context window
refreshes.
Always be as persistent and autonomous as possible and complete tasks
fully, even if the end of your budget is approaching.
Never artificially stop any task early regardless of the context
remaining.
</context_management>
```

## Behavior Control

### 4. Hyper-Proactive Mode

**Use this if:** You want Claude to take initiative and implement ideas without asking

```
<default_to_action>
By default, implement changes rather than only suggesting them.
If my intent is unclear, infer the most useful likely action and
proceed, using tools to discover any missing details instead of
guessing. When you come up with good ideas during implementation,
just implement them — you don't need to ask me first. I'll review
and approve afterward.
Prioritize making progress over waiting for perfect instructions.
Take initiative while staying aligned with the project's goals.
</default_to_action>
```

### 5. Conservative Mode

**Use this if:** You want Claude to slow down and only do exactly what you ask

```
<do_not_act_before_instructions>
Do not jump into implementation or change files unless clearly
instructed to make changes. When my intent is ambiguous, default to
providing information, doing research, and giving recommendations
rather than taking action.
Only proceed with edits, modifications, or implementations when I
explicitly request them. Ask clarifying questions before making
significant architectural decisions.
</do_not_act_before_instructions>
```

> **Pro Tip:** These two are opposites — pick ONE based on your working style. Proactive for experienced developers who review fast. Conservative for learning or critical systems.

### 6. Complete Work Systematically

**Use this if:** Claude sometimes leaves tasks partially finished

```
<complete_work_systematically>
This is a very long task, so plan out your work clearly.
It is encouraged to spend your entire output context working on the
task — just make sure you don't run out of context with significant
uncommitted work.
Continue working systematically until you have completed the task.
Don't leave functions half-implemented or features partially built.
Finish each component fully before moving to the next.
</complete_work_systematically>
```

## Code Quality

### 7. Prevent Over-Engineering

**Use this if:** Claude creates too many helper files or adds unnecessary abstractions

```
<avoid_overengineering>
Avoid over-engineering. Only make changes that are directly requested
or clearly necessary. Keep solutions simple and focused.
Don't add features, refactor code, or make "improvements" beyond
what was asked. A bug fix doesn't need surrounding code cleaned up.
A simple feature doesn't need extra configurability.
Don't add error handling, fallbacks, or validation for scenarios
that can't happen. Trust internal code and framework guarantees.
Only validate at system boundaries (user input, external APIs).
```

```
Don't create helpers, utilities, or abstractions for one-time
operations. Don't design for hypothetical future requirements.
The right amount of complexity is the minimum needed for the
current task.
</avoid_overengineering>
```

### 8. Real Solutions, Not Test-Passing Hacks

**Use this if:** Claude sometimes hard-codes values or creates workarounds

```
<write_real_solutions>
Write high-quality, general-purpose solutions using standard tools.
Do not create helper scripts or workarounds to accomplish tasks
more efficiently.
Implement solutions that work correctly for all valid inputs, not
just test cases. Do not hard-code values or create solutions that
only work for specific test inputs.
Focus on understanding problem requirements and implementing correct
algorithms. Tests verify correctness — they don't define the solution.
If the task is unreasonable, infeasible, or if any tests are
incorrect, please inform me rather than working around them.
</write_real_solutions>
```

---

## Specialized

### 9. Better Frontend Design

**Use this if:** Claude creates generic-looking UIs

```
<frontend_aesthetics>
You tend to converge toward generic outputs. In frontend design,
this creates what users call the "AI slop" aesthetic. Avoid this:
make creative, distinctive frontends that surprise and delight.
Focus on:
- Typography: Choose fonts that are beautiful and distinctive.
  Avoid generic fonts like Arial and Inter.
- Color & Theme: Commit to a cohesive aesthetic. Use CSS variables.
  Dominant colors with sharp accents outperform timid palettes.
- Motion: Use animations for effects and micro-interactions.
  Prioritize CSS-only solutions.
- Backgrounds: Create atmosphere and depth rather than defaulting
  to solid colors. Layer gradients, geometric patterns, or effects.
Avoid: Overused font families, clichéd color schemes (purple
gradients on white), predictable layouts, cookie-cutter design.
</frontend_aesthetics>
```

### 10. Explain Everything (Learning Mode)

**Use this if:** You're learning and want Claude to teach as it builds

```
<explain_your_work>
After completing any task that involves tool use (reading files,
writing code, running commands), provide a clear summary of:
- What you just did
- Why you did it that way
- What the outcome was
This helps me understand your process and learn from your decisions.
</explain_your_work>
```

### 11. State Tracking for Long Projects

**Use this if:** Complex applications that take multiple sessions

```
<state_tracking>
For this project, maintain state across sessions using:
```

```
1. tests.json: Track all tests with status (passing/failing/not_started)
2. progress.txt: Notes about what's completed and what's next
3. Regular commits with clear messages documenting progress
Before starting work in a new session:
- Review progress.txt for where we left off
- Check tests.json for what's passing/failing
- Review recent git logs for recent changes
- Run integration tests before implementing new features
Tests should only grow, never shrink. Focus on incremental progress.
</state_tracking>
```

### 12. Vision Capabilities Reminder

**Use this if:** You want Claude to use image understanding for debugging/design

```
<vision_capabilities>
I have exceptional vision capabilities and can process multiple
images simultaneously. When you encounter:
- UI bugs: Screenshot it and paste it in the terminal
- Design inspiration: Screenshot reference designs for me to analyze
- Complex visual issues: Show me rather than describe
Paste images directly into the terminal — I'll analyze them and
act accordingly.
</vision_capabilities>
```

---

## Quick Setup

```
# Create CLAUDE.md in your project root
touch CLAUDE.md

# Paste your chosen templates
# Save the file — Claude follows them immediately
```

## Choosing Your Configuration

| Working Style | Recommended Templates |
|---|---|
| **Fast & autonomous** | Proactive (#4) + Parallel (#2) + Anti-Hallucination (#1) |
| **Careful & controlled** | Conservative (#5) + Anti-Hallucination (#1) + Explain (#10) |
| **Long projects** | Context Mgmt (#3) + State Tracking (#11) + Complete Work (#6) |
| **Frontend dev** | Frontend Design (#9) + Anti-Overengineering (#7) + Proactive (#4) |
| **Learning** | Explain (#10) + Conservative (#5) + Anti-Hallucination (#1) |

---

## Pro Tips

1. **Explain your motivation**, not just the task: "Build a calendar component" → "Build a calendar as the central hub — all notes, tasks, and projects integrate with it"
2. **Load context at session start**: "Take a look at the app and architecture. Understand deeply how it works. Ask me any questions."
3. **Avoid the word "think"** — it triggers Extended Thinking mode, burning more tokens. Use: consider, evaluate, assess, determine, analyze
4. **Commit frequently** — every commit becomes context Claude can reference in future sessions
5. **Use images for debugging** — a screenshot of a bug is worth 1,000 words of description

---